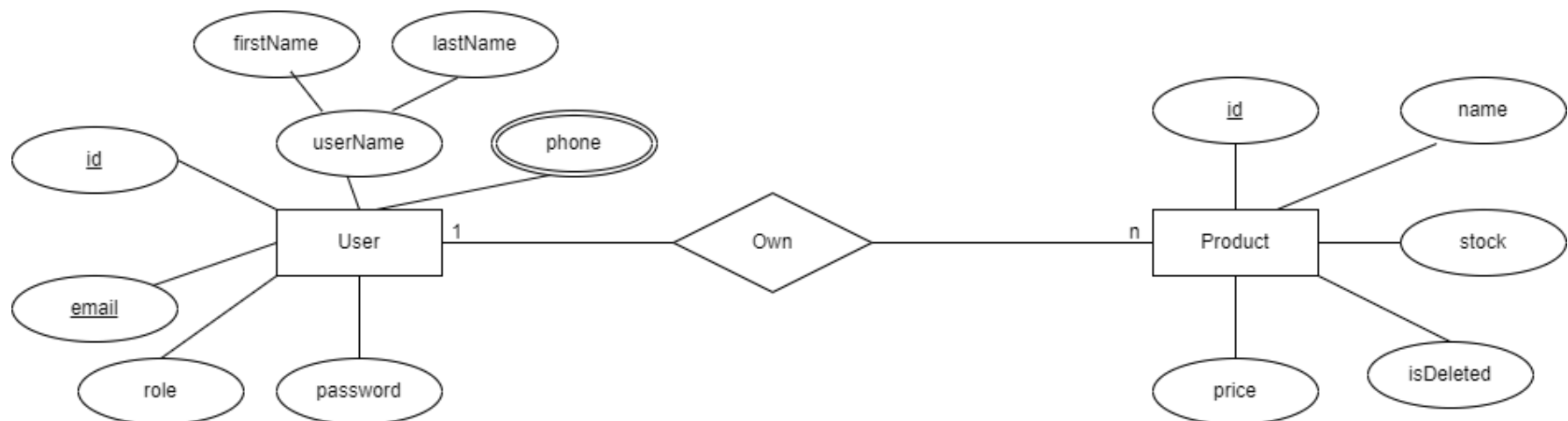


Part 1: ERD Diagram (3 Grade)

Musicana records have decided to store information on musicians who perform on their albums in a database. The company has wisely chosen to hire you as a database designer.

- Each musician that is recorded at Musicana has an ID number, a name, an address (street, city) and a phone number.
- Each instrument that is used in songs recorded at Musicana has a unique name and a musical key (e.g., C, B-flat, E-flat).
- Each album that is recorded at the Musicana label has a unique title, a copyright date, and an album identifier.
- Each song recorded at Musicana has a unique title and an author.
- Each musician may play several instruments, and a given instrument may be played by several musicians.
- Each album has a number of songs on it, but no song may appear on more than one album.
- Each song is performed by one or more musicians, and a musician may perform a number of songs.
- Each album has exactly one musician who acts as its producer.
- A producer may produce several albums.

Part2: Design a schema (Mapping) for the following ERD. (Use any design tool you want)
(2 Grade)



Part 3: (Using Node.js and MySQL) Answer the Questions below based on the given Scenario

The small retail store needs a RESTful API to manage information about its products, suppliers, and sales.

1. Products Table:

- **ProductID:** Unique identifier for each product (integer, primary key, auto-increment).
- **ProductName:** Name of the product (**text**).
- **Price:** Price of the product (**decimal**).
- **StockQuantity:** Quantity of the product in stock (**integer**).
- **SupplierID:** ID of the supplier providing the product (**integer, foreign key referencing Suppliers**).

2. Suppliers Table:

- **SupplierID:** Unique identifier for each product (integer, primary key, auto-increment).
- **SupplierName:** Name of the supplier (**text**).



Assignment4



- **ContactNumber:** Supplier's contact number (**text**).

3. Sales Table:

- **SaleID:** Unique identifier for each product (integer, primary key, auto-increment).
- **ProductID:** Reference to the product sold (**integer, foreign key referencing Products**).
- **QuantitySold:** Quantity of the product sold (**integer**).
- **SaleDate:** Date of sale (**date**).

(Using Node.js, Express.js, and MySQL) Build a RESTful API that performs the following tasks (15 Grades):

1. Configure a MySQL **connection pool** using **mysql2/promise**, create the required database (if it does not exist), and create the **Products**, **Suppliers**, and **Sales** tables with the appropriate primary and foreign key relationships. **(0.5 Grade)**
2. Create REST API endpoints to perform **CRUD operations** for the **Products** table: **(2.5 Grade)**
 - Create a product.
 - Retrieve all products.
 - Retrieve a product by ID.
 - Update a product.
 - Delete a product.
3. Create REST API endpoints to perform **CRUD operations** for the **Suppliers** table: **(2 Grade)**
 - Create a supplier.
 - Retrieve all suppliers.
 - Update supplier information.
 - Delete a supplier.
4. Create REST API endpoints to manage **Sales** : **(1.5 Grade)**
 - Record a sale.
 - Retrieve all sales.
 - Retrieve sales for a specific product.
5. Create API endpoints to perform the following database modifications: **(2 Grade)**
 - Add a **Category** column to the **Products** table.
 - Remove the **Category** column.
 - Change **ContactNumber** to **VARCHAR(15)**.
 - Add a **NOT NULL** constraint to **ProductName**.
6. **Create an API endpoint or initialization script to insert the following data:(1.5 Grade)**
 - a. Add a supplier with the name '**FreshFoods**' and contact number '**01001234567**'.
 - b. Insert the following three products, all provided by '**FreshFoods**':
 - i. '**Milk**' with a price of **15.00** and stock quantity of **50**.
 - ii. '**Bread**' with a price of **10.00** and stock quantity of **30**.
 - iii. '**Eggs**' with a price of **20.00** and stock quantity of **40**.
 - c. Add a record for the sale of **2** units of '**Milk**' made on '**2025-05-20**'.
7. Create an API endpoint to update the price of '**Bread**' to **25.00**. **(0.5 Grade)**
8. Create an API endpoint to delete the product '**Eggs**'. **(0.5 Grade)**
9. Create a reporting endpoint to retrieve the total quantity sold for each product using SQL aggregate functions. **(0.5 Grade)**
10. Create a reporting endpoint to retrieve the **product with the highest stock quantity**. **(0.5 Grade)**
11. Create a reporting endpoint to retrieve **suppliers whose names start with 'F'**. **(0.5 Grade)**
12. Create a reporting endpoint to retrieve **all products that have never been sold**. **(0.5 Grade)**
13. Create a reporting endpoint to retrieve **all sales** including: **(0.5 Grade)**
 - Product name
 - Quantity sold
 - Sale date using SQL **JOIN** operations.

Assignment4

14. Create a SQL script or secure administrative endpoint to create a MySQL user named **store_manager** and grant the following permissions on all tables: **(0.5 Grade)**
 - SELECT
 - INSERT
 - UPDATE
15. Revoke the **UPDATE** permission from “**store_manager**”. **(0.5 Grade)**
16. Grant **DELETE** permission to “**store_manager**” only on the Sales table. **(0.5 Grade)**

Bonus (2 Grades)

How to deliver the bonus?

- 1- Solve the problem [Customer Who Visited but Did Not Make Any Transactions](#) on **LeetCode**
- 2- Inside your assignment folder, create a **SEPARATE FILE** and name it “bonus.txt”
- 3- Copy the code that you have submitted on the website inside “bonus.txt” file.